Convertir les entiers

État des variables, divisions successives, base 16 et Python

Séquence
SQ01

Partie
P02

Niveau
Première NSI

Objectifs

- **Tracer** l'état des variables d'une boucle ou d'une fonction dans un tableau de suivi.
- **Convertir** un entier positif entre les bases 2, 10 et 16 par une méthode explicite.
- **Utiliser** `0b`, `0x`, `bin`, `hex` et `int` en Python sans confondre texte et entier.

Prérequis : poids des bits (partie 1), `lire_octet` et le travail à distance « Ce que la boucle retient », `//` et `%`.

Ce que la boucle retenait

`lire_octet("10000001")`, tracé au tableau :

Tour	`bit`	`total`	`poids`
avant	—	0	128
1	`"1"`	128	64
2 à 7	`"0"`	128	64 → 1
8	`"1"`	129	0

- `total` **accumule** les poids des bits à 1.
- `poids` **change à chaque tour**, que le bit vaille 0 ou 1.
- Le `if` décide seulement **si l'on ajoute**.

L'état d'un programme

L'**état**, à un instant donné, c'est la valeur de **chacune** des variables.

Règle du tableau de suivi	Pourquoi
Une colonne par variable	On voit tout ce que le programme garde en mémoire
Une ligne par tour	On voit l'évolution, pas seulement le résultat
Première ligne : état **avant** la boucle	L'initialisation fait partie du calcul
Valeurs **à la fin** du tour	Une convention unique, pas d'ambiguïté

Tracer d'abord, exécuter ensuite : l'exécution dit *quel* résultat, le tableau dit *pourquoi*.

ACTIVITÉ

Exercice 2.1 — Une boucle qui divise

Papier · seul puis à deux · 10 min

À produire : le tableau de suivi de `ecrire_binaire(13)`, colonnes `reste`, `bits`, `n`.

1. Tracer sans exécuter (handout, exercice 2.1).
2. Comparer avec son voisin, puis vérifier avec 13 = 8 + 4 + 1.
3. Questions 3 et 4 : et si on écrivait `bits + str(reste)` ? et pour `n = 0` ?

La fonction tracée

```
def ecrire_binaire(n):
    bits = ""
    while n > 0:
        reste = n % 2
        bits = str(reste) + bits
        n = n // 2
    return bits
```

- `n % 2` : le reste, 0 ou 1.
- `n // 2` : le quotient, qui devient le nouveau `n`.
- `str(reste) + bits` : le nouveau chiffre se place **devant**.
- La boucle s'arrête quand `n` vaut 0 — attention au cas `n = 0` dès le départ.

ACTIVITÉ

Exercice 2.2 — L'état dans une fonction

Papier puis console · seul · 5 min

À produire : les deux lignes affichées, puis une phrase qui explique la première.

```
def moitie(n):
    n = n // 2
    return n

n = 26
m = moitie(n)
print(n, m)
m = moitie(m)
print(n, m)
```

À RETENIR

Variables locales

1. Le paramètre `n` de `moitie` est une **autre variable** que le `n` du programme.
2. À l'appel, il reçoit une **copie de la valeur** ; il disparaît au `return`.
3. Seule la **valeur renvoyée** sort de la fonction, à condition de la ranger : `m = moitie(n)`.
4. Affichage : `26 13`, puis `26 6`.

Une méthode qui marche toujours

Les divisions successives

Division	Quotient	Reste
$13 \div 2$	6	**1**
$6 \div 2$	3	**0**
$3 \div 2$	1	**1**
$1 \div 2$	0	**1**

1. Diviser par 2, noter le **reste**, garder le **quotient**.
2. Recommencer jusqu'à un quotient nul.
3. Lire les restes **de bas en haut** : $13 = \mathbf{1101}_2$.

C'est exactement `ecrire_binaire`, à la main.

Pourquoi lire à l'envers ?

- Le premier reste dit si le nombre est **pair ou impair** : c'est le bit de poids 1, à droite.
- Diviser par 2 **décale** l'écriture d'un cran vers la droite : 1101 → 110 → 11 → 1.
- Chaque division fait donc « tomber » le bit suivant, de la droite vers la gauche.
- Même méthode dans **toute base b** : diviser par b, les restes vont de 0 à b – 1.

Pour comparer, en base 10 : 473 ÷ 10 → reste **3** ; 47 ÷ 10 → reste **7** ; 4 ÷ 10 → reste **4**. Lus de bas en haut : 4, 7, 3.

La base 16

Seize chiffres

Un octet en binaire, c'est huit chiffres : long à écrire, facile à mal recopier.

Base 10	**0–9**	**10**	**11**	**12**	**13**	**14**	**15**
Base 16	0–9	A	B	C	D	E	F
Base 2	0000–1001	1010	1011	1100	1101	1110	1111

Poids : 1, 16, 256, 4096… Exemple : `2F` = 2 × 16 + 15 = **47**.

Un chiffre hexadécimal = quatre bits

16 = 2^4 : chaque chiffre hexadécimal correspond **exactement** à un quartet.

Quartet	1011	0110
Lecture 8-4-2-1	11	6
Chiffre	B	6

1. Découper en paquets de 4 bits **à partir de la droite**.
2. Lire chaque paquet comme `lire_quartet`.
3. Assembler : `10110110` = **B6**.

Chemin inverse : chaque chiffre redonne ses 4 bits.

L’hexadécimal autour de vous

Usage	Exemple	Ce qui est codé
Couleur web (CSS)	`#FF8800`	3 octets : rouge 255, vert 136, bleu 0
Adresse MAC	`3C:52:82:1A:F0:07`	6 octets identifiant une carte réseau
Adresse IPv6	`2001:db8::1`	16 octets, par groupes de 2 octets
Éditeur hexadécimal	`89 50 4E 47`	Les premiers octets d’un fichier PNG

Partout où l’on manipule des octets, l’hexadécimal remplace le binaire pour les humains.

Les entiers en Python

Trois écritures, un seul nombre

```
>>> 0b1101
13
>>> 0x2F
47
>>> 0b1101 == 13 == 0xD
True
```

- `0b` : écriture binaire d'un entier.
- `0x` : écriture hexadécimale d'un entier.
- La machine stocke **toujours** du binaire : seule l'écriture change.
- `13`, `0b1101` et `0xD` sont le **même** objet `int`.

Convertir : dans quel sens ?

Écriture	Résultat	Type	Sens
`bin(13)`	`'0b1101'`	`str`	entier → texte
`hex(47)`	`'0x2f'`	`str`	entier → texte
`int("1101", 2)`	`13`	`int`	texte → entier
`int("2F", 16)`	`47`	`int`	texte → entier

On **calcule** sur des entiers ; on convertit en texte pour **afficher**.

ATTENTION

Texte ou entier ?

```
>>> bin(5) + 1
TypeError: can only concatenate str (not "int") to str
>>> int(bin(5), 2) + 1
6
```

- `bin(5)` est le **texte** `'0b101'` : on ne lui ajoute pas un nombre.
- `int` accepte le préfixe : `int("0b101", 2)` vaut 5.
- Réflexe : `type(...)` en cas de doute.

Des entiers sans limite

Langage ou machine	Taille d'un entier	Plus grand entier positif
Machine des nombres	8 bits	255
Java `int`, C `int32_t`	32 bits	2 147 483 647
Java `long`, C `int64_t`	64 bits	environ $9{,}2 \times 10^{18}$
Python `int`	variable	limité par la mémoire

`2 ** 100` s'affiche en entier : Python réserve autant de bits que nécessaire. La partie 3 montre ce que coûte une taille fixe.

ACTIVITÉ

Exercice 2.3 — Écrire `ecrire_binaire`

Sur ordinateur · seul · 15 min — page du cours (Python dans la page) ou EduPython

À produire : une fonction qui passe les 7 tests, **sans** `bin` ni `format`.

1. Sans regarder l'exercice 2.1 : tracer d'abord `ecrire_binaire(6)` sur papier.
2. Traiter `n == 0` à part pour renvoyer `"0"`.
3. Exécuter les tests ; si un test échoue, retracer ce cas.

ACTIVITÉ

Exercice 2.4 et 2.5 — Convertir, prévoir

Papier puis console · seul · 15 min

À produire : les quatre conversions avec la méthode, puis les quatre réponses du QCM.

1. 45 en base 2 · `11010011` en base 16 · `0x3A` en bases 2 et 10.
2. `#1E90FF` : valeur décimale de chaque octet.
3. QCM 2.5 : prévoir **avant** de vérifier dans la console.

ACTIVITÉ

Exercice 2.6 — Pour aller plus loin

Sur ordinateur · seul · pour ceux qui ont fini

À produire : `ecrire_hexa(n)` qui passe les tests, dont `ecrire_hexa(48879) == "BEEF"`.

1. Reprendre la structure de `ecrire_binaire` avec `// 16` et `% 16`.
2. Le reste sert de position dans `"0123456789ABCDEF"`.

À RETENIR

À retenir

1. Un **tableau de suivi** : une colonne par variable, une ligne par tour, l'état avant la boucle.
2. Les variables d'une fonction sont **locales** : seule la valeur renvoyée en sort.
3. **Divisions successives** par b : les restes, lus du dernier au premier, donnent l'écriture en base b.
4. Un chiffre hexadécimal = **quatre bits**.
5. `0b` , `0x` écrivent un entier ; `bin` , `hex` donnent du texte ; `int(texte, base)` redonne un entier.